GOmpiler - A C++ Compiler Written in Go

Abstract

GOmpiler is an advanced C++ compiler implemented in Go, demonstrating the application of modern compiler construction principles and techniques. This project delivers a complete compilation pipeline that transforms C++ source code into executable binaries through multiple sophisticated stages including lexical analysis, parsing, semantic analysis, intermediate representation generation, optimization, and linking.

The compiler implements a lexical analyzer using deterministic finite automata principles to tokenize source code, a recursive descent parser to construct Abstract Syntax Trees (AST), and comprehensive semantic analysis for type checking and scope resolution. The intermediate representation (IR) generation facilitates code optimization, while the linking phase handles symbol resolution and generates standard ELF binaries.

GOmpiler's modular architecture enables clear separation of concerns across the compilation pipeline, making it both an educational tool for understanding compiler design and a practical demonstration of implementing complex language processing systems in Go. The project leverages Go's strengths in concurrency and memory management while supporting standard C++ features.

This implementation represents a cross-language approach to compiler construction, bridging the gap between modern systems programming (Go) and traditional compiled languages (C++), demonstrating the versatility of compilation techniques across different programming paradigms.

Authors

- Dinesh TV (22PT09)
- Mohamed Aashir S (22PT17)

Technologies

- Go 1.24
- LLVM toolchain (for final compilation steps)
- ELF binary format